

تعلم Python

من الصفر إلى الاحتراف

دليل عربي عملي مع أمثلة كاملة وتمارين بعد كل فصل

إعداد: Manus AI

الإصدار الأول – 2026

الفهرس

1	كيف تستخدم هذا الكتاب	1
1	الفصل الأول: الأساسيات	2
1	1.1 أول برنامج ومتغيراتك	2.1
1	1.2 الإدخال والعمليات الحسابية	2.2
2	1.3 الشروط واتخاذ القرار	2.3
2	1.4 الحلقات التكرارية	2.4
2	2.5 تمارين الفصل الأول	2.5
3	الفصل الثاني: هياكل البيانات	3
3	2.1 القوائم Lists	3.1
3	2.2 الصفوف والقواميس والمجموعات	3.2
3	2.3 التقطيع والفهم الاستيعابي	3.3
3	3.4 تمارين الفصل الثاني	3.4
4	الفصل الثالث: الدوال والوحدات	4
4	3.1 كتابة دالة قابلة لإعادة الاستخدام	4.1
4	3.2 النطاق والاستثناءات	4.2
4	3.3 الوحدات Modules	4.3
5	4.4 تمارين الفصل الثالث	4.4
5	الفصل الرابع: البرمجة الكائنية	5
5	4.1 الصنف والكائن	5.1
5	4.2 التغليف والوراثة	5.2
6	5.3 تمارين الفصل الرابع	5.3
6	الفصل الخامس: التعامل مع الملفات	6
6	5.1 القراءة والكتابة بأمان	6.1
6	5.2 JSON لحفظ البيانات المنظمة	6.2
6	6.3 تمارين الفصل الخامس	6.3
7	الفصل السادس: مشاريع عملية	7
7	6.1 مشروع: مدير مهام بسيط	7.1
8	6.2 مشروع: محلل نص قصير	7.2
8	7.3 تمارين الفصل السادس	7.3
8	خريطة الاحتراف التالية	8

1 كيف تستخدم هذا الكتاب

Python لغة برمجة واضحة وقوية. ستتعلم هنا بطريقة «اقرأ ثم جرّب»: كل فكرة تتبعها قطعة كود كاملة، ثم تمرين صغير. لا تحاول حفظ جميع التفاصيل؛ انسخ المثال، شغله، ثم غير قيمة واحدة على الأقل ولاحظ النتيجة. < القاعدة الذهبية: الخطأ ليس علامة فشل، بل رسالة تخبرك بما تحتاج إلى تصحيحه. للتنشغيل ثبّت Python من موقعه الرسمي أو استخدم محرر المنصة المرفقة. احفظ برامجك بامتداد py. ثم شغلها من الطرفية بالأمر py. اسم الملف python.

2 الفصل الأول: الأساسيات

2.1 1.1 أول برنامج ومتغيراتك

الدالة print تعرض نصاً أو قيمة على الشاشة. المتغير اسم يشير إلى قيمة؛ لا تحتاج في Python إلى إعلان نوع المتغير مقدماً. استخدم أسماء معبّرة مثل student_name بدلاً من x عندما يكون المعنى مهماً.

```
# hello.py
name = "سارة"
age = 18
is_student = True

print("مرحباً", name)
print("العمر بعد سنة:", age + 1)
print("هل تدرس؟", is_student)
```

القيم الأساسية هي: النصوص str مثل "مرحبا"، والأعداد الصحيحة int مثل 7، والعشرية float مثل 3.5، والمنطقية bool التي تكون True أو False. استخدم type عندما تريد معرفة النوع.

```
price = 19.95
print(type(price))      # <class 'float'>
print(type("Python"))   # <class 'str'>
```

2.2 1.2 الإدخال والعمليات الحسابية

الدالة input تقرأ النص من المستخدم دائماً كنص. إن أردت عدداً، حوّل به int أو float. انتبه إلى أن القسمة العادية / تعطي عدداً عشرياً، والقسمة الصحيحة // تحذف الجزء العشري، و% يعيد الباقي.

```
name = input("اكتب اسمك: ")
hours = float(input("كم ساعة درست اليوم؟"))
weekly_goal = 10
remaining = max(0, weekly_goal - hours)

print(f"أحسن يا {name}.")
print(f"ساعات {remaining} تبقى من هدفك الأسبوعي")
```

```
number = 17
print(number / 5)      # 3.4
print(number // 5)     # 3
print(number % 5)      # 2
print(number ** 2)     # 289
```

2.3 1.3 الشروط واتخاذ القرار

تسمح الجملة if للكود باختيار مسار. يجب أن تكون المسافة البادئة داخل الشرط ثابتة، وغالباً أربع مسافات. استخدم elif لشرط إضافي و else للحالة المتبقية.

```
score = int(input("اكتب درجتك: "))

if score >= 90:
    level = "ممتاز"
elif score >= 75:
    level = "جيد جداً"
elif score >= 60:
    level = "ناجح"
else:
    level = "يحتاج إلى مراجعة"

print("التقدير:", level)
```

يمكنك تركيب الشروط باستخدام and و or و not:

```
age = 20
has_ticket = True
can_enter = age >= 18 and has_ticket
print(can_enter)
```

2.4 1.4 الحلقات التكرارية

استخدم for عندما تعرف عدد التكرارات أو تريد المرور على عناصر، واستخدم while عندما يستمر التكرار حتى يتحقق شرط. لا تنس تحديث قيمة الشرط في while حتى لا تنشئ حلقة لا نهائية.

```
for lesson in range(1, 6):
    print(f"ابدأ الدرس رقم {lesson}")

attempts = 3
while attempts > 0:
    password = input("اكتب كلمة المرور: ")
    if password == "python":
        print("تم الدخول")
        break
    attempts -= 1
    print("حاول مرة أخرى")
else:
    print("انتهت المحاولات")
```

2.5 تمارين الفصل الأول

1. اكتب برنامجاً يطلب طول المستطيل وعرضه ثم يطبع المساحة والمحيط.
2. اكتب برنامجاً يقرأ درجة حرارة ويطبع «بارد» إذا كانت أقل من 15، و«معتدل» إذا كانت بين 15 و 29، و«حار» في غير ذلك.
3. اطبع الأعداد الزوجية من 2 إلى 20 باستخدام range.

3 الفصل الثاني: هياكل البيانات

3.1 2.1 القوائم Lists

القائمة تحفظ مجموعة مرتبة قابلة للتعديل. تبدأ الفهارس من الصفر؛ لذا tasks[0] هو أول عنصر. يمكنك إضافة عنصر بـ append، وإزالة قيمة بـ remove، وفرز القائمة بـ sort.

```
tasks = ["قراءة", "تدريب", "مراجعة"]
tasks.append("مشروع صغير")
tasks[1] = "Python تدريب"

for index, task in enumerate(tasks, start=1):
    print(f"{index}. {task}")

tasks.remove("مراجعة")
print(tasks)
```

3.2 2.2 الصفوف والقواميس والمجموعات

الصف tuple يشبه القائمة لكنه غير قابل للتعديل؛ مناسب لإحداثيات ثابتة. القاموس dict يخزن أزواج مفتاح وقيمة، وهو ممتاز لوصف طالب أو منتج. المجموعة set تحفظ قيماً فريدة دون تكرار.

```
location = (24.7136, 46.6753)
student = {
    "name": "ليان",
    "level": "مبتدئ",
    "completed": 4,
}
skills = {"python", "html", "python", "css"}

student["completed"] += 1
print(student["name"], student["completed"])
print(skills) # لا تكرر python
print(location[0])
```

3.3 2.3 التقطيع والفهم الاستيعابي

التقطيع slice يأخذ جزءاً من سلسلة أو قائمة. التعبير الاستيعابي للقوائم يصنع قائمة جديدة بوضوح من تكرار قصير.

```
numbers = [3, 8, 1, 6, 10]
print(numbers[1:4]) # [8, 1, 6]
print(numbers[::-1]) # نسخة معكوسة

squares_of_even = [n ** 2 for n in numbers if n % 2 == 0]
print(squares_of_even) # [64, 36, 100]
```

3.4 تمارين الفصل الثاني

1. أنشئ قائمة بأسماء خمسة كتب وأطبع الأول والأخير ثم أضف كتاباً جديداً.
2. أنشئ قاموساً لمنتج يحتوي الاسم والسعر والكمية، ثم احسب قيمة المخزون.
3. اكتب برنامجاً يحول قائمة درجات إلى قائمة تحتوي الدرجات الناجحة فقط (60 فأعلى).

4 الفصل الثالث: الدوال والوحدات

4.1 3.1 كتابة دالة قابلة لإعادة الاستخدام

الدالة تجمع منطقاً يؤدي مهمة واحدة. المعاملات هي مدخلاتها، و `return` يعيد الناتج. اكتب اسماً فعلياً واضحاً وقُلِّل اعتماد الدالة على المتغيرات الخارجية.

```
def calculate_discount(price, percent=0):
    """تعيد السعر بعد خصم النسبة المئوية"""
    if not 0 <= percent <= 100:
        raise ValueError("نسبة الخصم يجب أن تكون بين 0 و100")
    return price * (1 - percent / 100)

final_price = calculate_discount(120, 15)
print(f"السعر النهائي: {final_price:.2f}")
```

4.2 3.2 النطاق والاستثناءات

المتغير داخل الدالة محلي غالباً، فلا يظهر خارجها. الاستثناءات تمنع انهيار البرنامج عندما يأتي إدخال غير صالح. التقط فقط الأخطاء التي تعرف كيف تعالجها.

```
def read_positive_number():
    while True:
        try:
            value = float(input("اكتب رقماً موجباً: "))
            if value <= 0:
                raise ValueError
            return value
        except ValueError:
            print("الرجاء إدخال رقم أكبر من صفر.")

print(read_positive_number())
```

4.3 3.3 الوحدات Modules

الوحدة ملف Python يمكن استيراد وظائفه. مكتبة `math` مثال جاهز. لتبني وحدتك، أنشئ ملفاً باسم `helpers.py` وضع فيه دوالاً، ثم استوردها من ملف آخر.

```
# helpers.py
def is_even(number):
    return number % 2 == 0

# app.py
from helpers import is_even
import math

print(is_even(12))
print(math.sqrt(81))
```

4.4 تمارين الفصل الثالث

1. اكتب دالة is_palindrome تتحقق هل الكلمة تقرأ بالشكل نفسه من اليمين واليسار بعد تحويلها إلى حروف صغيرة.
2. اكتب دالة تستقبل قائمة أرقام وتعيد أكبر قيمة وأصغر قيمة في قاموس واحد.
3. أضف معالجة خطأ لبرنامج قسمة عددين بحيث لا يسمح بالقسمة على صفر.

5 الفصل الرابع: البرمجة الكائنية

5.1 4.1 الصنف والكائن

الصنف قالب، والكائن نسخة منه. تضع `__init__` القيم الأولية للكائن، وتمثل `self` الكائن الحالي. استخدم الخصائص للبيانات والطرائق للسلوك.

```
class Course:
    def __init__(self, title, lessons):
        self.title = title
        self.lessons = lessons
        self.completed = 0

    def complete_lesson(self):
        if self.completed < self.lessons:
            self.completed += 1

    def progress(self):
        return self.completed / self.lessons * 100

python_course = Course("Python", 12)
python_course.complete_lesson()
print(f"{python_course.title}: {python_course.progress():.0f}%")
```

5.2 4.2 التغليف والوراثة

لا تمنع الشرطة السفلية المزدوجة الوصول تقنياً، لكنها تشير إلى خاصية داخلية. الوراثة تسمح لصنف متخصص بإعادة استخدام صنف عام. لا تستخدمها إلا إذا كانت العلاقة «نوع من» منطقية.

```
class User:
    def __init__(self, name):
        self.name = name

    def greeting(self):
        return f"مرحباً {self.name}"

class Student(User):
    def __init__(self, name, track):
        super().__init__(name)
        self.track = track

    def greeting(self):
        return f"{super().greeting()}، أنت في مسار {self.track}"
```

```
student = Student("عمر", "Python")
print(student.greeting())
```

5.3 تمارين الفصل الرابع

1. صمّم صنف BankAccount فيه إيداع وسحب ورصيد، ولا تسمح بالسحب فوق الرصيد.
2. صمّم صنف Book ثم صنف EBook يرث منه ويضيف خاصية حجم الملف.
3. أضف طريقتي __str__ إلى أحد الأصناف لتطبع وصفاً مفهوماً للكائن.

6 الفصل الخامس: التعامل مع الملفات

6.1 5.1 القراءة والكتابة بأمان

استخدم with لفتح الملف، فهي تغلقه تلقائياً حتى لو حدث خطأ. الوضع w يكتب من جديد، و a يضيف في النهاية، و r يقرأ. حدد الترميز utf-8 للغة العربية.

```
notes = ["تعلمت القوائم", "أنشأت دالة", "سأبني مشروعاً"]

with open("notes.txt", "w", encoding="utf-8") as file:
    for note in notes:
        file.write(note + "\n")

with open("notes.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())
```

6.2 5.2 JSON لحفظ البيانات المنظمة

JSON صيغة شائعة لنقل البيانات. في Python استخدم json.dump للحفظ و json.load للقراءة. لا تستخدم eval أبداً لقراءة بيانات ملف غير موثوق.

```
import json

profile = {"name": "ندى", "tracks": ["Python", "HTML"], "points": 120}

with open("profile.json", "w", encoding="utf-8") as file:
    json.dump(profile, file, ensure_ascii=False, indent=2)

with open("profile.json", "r", encoding="utf-8") as file:
    loaded_profile = json.load(file)

print(loaded_profile["tracks"])
```

6.3 تمارين الفصل الخامس

1. أنشئ ملفاً يومية باسم journal.txt وأضف إليه سطرًا من المستخدم في كل تشغيل دون حذف السطور السابقة.
2. احفظ قائمة مهام في JSON ثم اقرأها واطبع المهام غير المكتملة.
3. عالج خطأ FileNotFoundError عند محاولة قراءة ملف غير موجود.

7 الفصل السادس: مشاريع عملية

7.1 6.1 مشروع: مدير مهام بسيط

يجمع المشروع القوائم والدوال والملفات. يبدأ بمهام في الذاكرة، يتيح الإضافة والإكمال، ثم يحفظها في ملف JSON. ابدأ بهذا الإصدار، ثم أضف البحث أو تاريخ الاستحقاق بنفسك.

```
import json
from pathlib import Path

FILE_NAME = Path("tasks.json")

def load_tasks():
    if FILE_NAME.exists():
        return json.loads(FILE_NAME.read_text(encoding="utf-8"))
    return []

def save_tasks(tasks):
    FILE_NAME.write_text(
        json.dumps(tasks, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

def show_tasks(tasks):
    if not tasks:
        print("لا توجد مهام بعد.")
        return
    for i, task in enumerate(tasks, start=1):
        mark = "✓" if task["done"] else "."
        print(f"{i}. [{mark}] {task['title']}")

tasks = load_tasks()
while True:
    print("\n1: عرض 2: إضافة 3: إكمال 4: خروج")
    choice = input("اختر: ")
    if choice == "1":
        show_tasks(tasks)
    elif choice == "2":
        title = input("اسم المهمة: ").strip()
        if title:
            tasks.append({"title": title, "done": False})
            save_tasks(tasks)
    elif choice == "3":
        show_tasks(tasks)
    try:
        index = int(input("رقم المهمة: ")) - 1
        tasks[index]["done"] = True
        save_tasks(tasks)
    except (ValueError, IndexError):
```

```
print("رقم غير صحيح.")
elif choice == "4":
    break
```

7.2 6.2 مشروع: محلل نص قصير

```
def analyze_text(text):
    words = text.split()
    letters = sum(char.isalpha() for char in text)
    return {
        "characters": len(text),
        "letters": letters,
        "words": len(words),
        "longest_word": max(words, key=len, default=""),
    }

text = input("اكتب فقرة: ")
for label, value in analyze_text(text).items():
    print(f"{label}: {value}")
```

7.3 تمارين الفصل السادس

1. وشع مدير المهام بإضافة أمر لحذف مهمة وحقل «تاريخ استحقاق».
2. اكتب لعبة تخمين رقم تستخدم random.randint وتعرض عدد المحاولات.
3. ابن برنامجاً يسجل مصروفاتك في JSON ويحسب مجموع كل فئة.

8 خريطة الاحتراف التالية

أنت الآن تمتلك أساساً عملياً: أنواع البيانات، التحكم بالتدفق، الدوال، الكائنات، الملفات، ومشروعات صغيرة. الخطوة التالية ليست قراءة عشر مكتبات جديدة؛ بل بناء مشروع واحد ورفعته إلى GitHub. بعد ذلك تعلم بيئة افتراضية venv، والاختبارات بـ pytest، واستهلاك واجهات API، ثم إطار ويب مثل Flask أو Django عندما تحتاجه. < تدرب يومياً 20 دقيقة: حل تمرين واحد، غيّر مثلاً واحداً، وسجل ما فهمته. الاستمرارية أقوى من جلسة طويلة متباعدة.